A reformulation of the Eta Type System

Thomas Lu (netid:cl2625)

May 13, 2026

Introducing the Nothing type

We have designed a syntax directed scheme for type derivation that is fully compliant with the formalization of the Eta type system. However, there are two issues that prevent us from directly adopting the formalization as our type derivation scheme. The first is determining the type of the lvalue _ (wildcard). As the specification has indicated, this could be easily done by adding a "special symbol" that is structurally equal to all other types. The other issue is determining the type of the empty array initializer expression {}. In the original formalization of the type system, when the metavariable $n = 0$, the inference rule for array initializer expressions reduces to the following axiom:

$$\frac{}{\Gamma \vdash \{\} : t[]}\ \textsc{emptyArray}$$

This axiom thus prevents us from using the formalization directly for syntax-directed type derivation, because the type of {} is undetermined when it is first visited in the AST. Furthermore, this issue cannot be as easily resolved by introducing a "special symbol" similar to the first issue, since the following expressions are all well-typed in the formalization:

$$\{\}[0], \quad \{\}[0] + 1, \quad \{\}[0][0][0], \quad \{\{\{\}\}\} + \{\{\}\}$$

As a result, a nontrivial reformulation of the type system of expressions is required to achieve syntax-directed type derivation. To begin, we say that the element type of an empty array initializer is empty, which represents an "unknown" type. Hence, in our reformulated type system where judgements have form $\Gamma \vdash e :: t$, we have the following axiom:

$$\frac{}{\Gamma \vdash \{\} :: \mathsf{empty}[]}\ \textsc{emptyArray}$$

Now we need to determine how the types are compared in the new type system. In an ordinary syntax-directed type system, when we want to determine whether two terms could be used in polymorphic equality, we would just derive their types and compare the types for exact structural equality. However, this no longer works in our reformulated system because $\{\} == \{1, 2, 3\}$ should be well-typed, even though the first array has type $\mathsf{empty}\,[]$ and the second $\mathsf{int}\,[]$. This suggests that we need to introduce a subtyping relation $\leq$ to compare types. Since (almost any useful) subtyping relation is a partial order, we immediately obtain the follwing axiom regarding reflexivity:

$$\frac{}{\mathsf{int} \leq \mathsf{int}}\ \textsc{IntRef} \qquad \frac{}{\mathsf{bool} \leq \mathsf{bool}}\ \textsc{BoolRef}$$

A (useful) subtyping relation is also preserved by taking array types:

$$\frac{t_1 \leq t_2}{t_1\,[] \leq t_2\,[]}\ \textsc{Array}$$

Now we decide the behavior of the newly introduced empty. Intuitively, if $t_1 \leq t_2$ in a subtyping relation, then a term of t_1 can be used whenever a term of t_2 is expected. Since the type empty can be substituted with any type in the original formalization (for example, to typecheck the expression $\{\} == \{1, 2, 3\}$, we substitute $t = \mathsf{int}$ in emptyArray and derive $\Gamma \vdash \{\} : \mathsf{int}\,[]$), we must have the following axiom:

$$\frac{}{\mathsf{empty} \leq t}\ \textsc{Nothing}$$

which completes the definition of the relation [1]. We observe that this relation is computable for any pairs of types. In a formalized subtyped type system, the following rule is used to widen types during type checking:

$$\frac{\Gamma \vdash e : t \quad t \leq t'}{\Gamma \vdash e : t'} \text{ WIDENING}$$

But again this rule breaks syntax-directedness since using this rule requires us to determine which type t should be widened to as soon as we derived the type of e. Hence, we need to "embed" the widening actions into the typing judgements of expressions in a deterministic way. This requires us to compute the "least general" supertype $t_1 \vee t_2$ of two types t_1, t_2, if it exists at all, so that we don't "overly" widen types (for example, if we widen $\{\} : \mathsf{empty}[]$ to $\{\} : \mathsf{int}[][]$, then we can't typecheck $\{\} + \{1, 2, 3\}$). Luckily for this type system, if $t_1 \vee t_2$ exists then it must be one of t_1 or t_2, so we can simply compute $t_1 \vee t_2$ by checking $t_1 \leq t_2$ and $t_2 \leq t_1$. We use $\overline{t_1 \vee t_2}$ to denote that $t_1 \vee t_2$ exists.

We are finally ready to reformulate the type system. The axioms of the literals remain the same. We will first see the $\vee$ operator in action with the polymorphic equality and inequality operators:

$$\frac{\Gamma \vdash e_1 : t_1 \quad \Gamma \vdash e_2 : t_2 \quad \overline{t_1 \vee t_2} \quad \leftrightarrow \in \{==, !=\}}{\Gamma \vdash e_1 \leftrightarrow e_2 : \mathsf{bool}}$$

For the rules in the original type system that expects a concrete, atomic type, we simply compare that type against the type of our new judgment with the subtyping relation, for example:

$$\frac{\Gamma \vdash e_1 :: t_1 \quad \Gamma \vdash e_2 :: t_2 \quad t_1 \leq \mathsf{int} \quad t_2 \leq \mathsf{int}}{\Gamma \vdash e_1 * e_2 :: \mathsf{int}} \text{ Ex1-Binary} \qquad \frac{\Gamma \vdash e :: t \quad t \leq \mathsf{bool}}{\Gamma \vdash !e_1 :: \mathsf{bool}} \text{ Ex2-Unary}$$

Rules involving arrays are a bit trickier. For example, for the typing rule of $\mathsf{length}(e)$, we need to check that the type t of e could be widened to some array type. Crucially, we observe that the least general array type t should be widened to is equal to $t \vee \mathsf{empty}[]$. Hence, we obtain the following typing rule:

$$\frac{\Gamma \vdash e :: t \quad \overline{t \vee \mathsf{empty}[]}}{\Gamma \vdash \mathsf{length}(e) :: \mathsf{int}}$$

The rule for array indexing is slightly more complex because we need to fetch the element type of the widened array type:

$$\frac{\Gamma \vdash e_1 :: t_1 \quad t_1'[] = t_1 \vee \mathsf{empty}[] \quad \Gamma \vdash e_2 :: t_2 \quad t_2 \leq \mathsf{int}}{e_1[e_2] :: t_1'}$$

Additionally, for array addition, after we widened the types of the expressions to array types, we also need to compute the least supertype the widened array types to determine the final output type.

$$\frac{\Gamma \vdash e_1 :: t_1 \quad t_1'[] = t_1 \vee \mathsf{empty}[] \quad \Gamma \vdash e_2 :: t_2 \quad t_2' = t_2 \vee \mathsf{empty}[] \quad t = t_1' \vee t_2'}{\Gamma \vdash e_1 + e_2 :: t[]}$$

The remaining reformulations about array initializers and function applications should become clear with the explanation above.

$$\frac{\Gamma \vdash e_1 :: t_1 \quad ..... \quad \Gamma \vdash e_n :: t_n \quad t = \bigvee_{i=1}^{n} t_i \quad n > 0}{\Gamma \vdash \{e_1, ..., e_n\} :: t[]}$$

$$\frac{\Gamma(f) = \mathsf{fn}(t_1, ..., t_n) \to (t') \quad \Gamma \vdash e_i : t_i^{\,(\forall i \in 1...n)} \quad t_i' \leq t_i^{\,(\forall i \in 1...n)}}{\Gamma \vdash f(e_1, .., e_n) :: t'}$$

We observe that these rules form a syntax-directed type system that is compliant with the original formulation of the type system.

[1] technically the relation is not complete, since we also need to incorporate the type all of the lvalue _ that is a supertype of any type, but doing so introduces some inconveniences that we wish to avoid in the Design Overview Document.

Finally, we note that the Eta programming language supports multiple declarations in a comma-separated declaration list, but a typing rule for this syntax is not found in the formalization of the type system. We provide typing rules for this syntax, generalized from the typing rules of single declarations in the same manner as multi-assignments:

$$\frac{x \notin \mathsf{dom}(\Gamma')}{\Gamma, \Gamma' \vdash x : t \dashv \Gamma'[x \mapsto \mathsf{var}(t)]} \qquad \frac{x \notin \mathsf{dom}(\Gamma') \quad \Gamma \vdash t_i \ {}^{(\forall i \in 1...n} \quad t_i \leq \mathsf{int} \ {}^{(\forall i \in 1...n} \quad t \in \{\mathsf{int}, \mathsf{bool}\} \quad n+m > 0}{\Gamma, \Gamma' \vdash x : t\,[e_1]...[e_n]\ \underbrace{[]...[]}_{m} \dashv \Gamma'[x \mapsto \mathsf{var}\ t\ \underbrace{[]...[]}_{n+m}]}$$

$$\frac{\Gamma_1 = \Gamma \quad \Gamma, \Gamma_i \vdash d_i \dashv \Gamma_{i+1} \ {}^{(\forall i \in 1...n}}{\Gamma \vdash d_1, ..., d_n \dashv \Gamma_{n+1}}$$

Extending Eta's Type System to Rho

In PA6, we are extending the Eta Programming Language to the Rho Programming Language, which includes constructs for records and break statements. This mean we need to extend the definition of the Eta AST for the source language as follows (new definitions are boxed):

$prog ::= gd_1\ gd_2\ ...gd_n$ (programs)

$gd ::=$ (global declarations)

- $f(x_1 : t_1, ..., x_n : t_n) : t'_1, ..., t_n\ \{s_1, ..., s_n\}$
- $x : t$
- $x : t = e_{\mathsf{lit}}$

$s ::=$ (statements)

- $\{s_1, ..., s_n\}$
- $\mathsf{if}\ e\ \mathsf{then}\ s_1$
- $\mathsf{if}\ e\ \mathsf{then}\ s_1\ \mathsf{else}\ s_2$
- $\mathsf{while}\ e\ \mathsf{do}\ s$
- $\boxed{\mathsf{break}}$
- $\mathsf{return}\ e_1\ ...\ e_n$
- $f(e_1, ..., e_n)$
- $x_1 : td_1, x_2 : td_2, ..., x_n : td_n$
- $lv_1, lv_2, ..., lv_n = e_1, ..., e_n$
- $lv_1, lv_2, ..., lv_n = f(e_1, ..., e_m)$

$td ::= \mathsf{int} \mid \mathsf{bool} \mid \mathsf{arr}(td, e) \mid \mathsf{arr}(td)$ (type specifiers in variable declarations)

- $\boxed{\mathsf{record}\ r}$

$lv ::= _ \mid x \mid x : t \mid e_1[e_2]$ (lvalues)

- $\boxed{e.x}$

$e ::=$ (expressions)

- e_{lit}
- str
- x
- $e_1 \odot e_2$
- $\square e$
- $e_1[e_2]$
- $\{e_1, ..., e_n\}$
- $\mathsf{length}(e)$
- $f(e_1, ..., e_n)$

$$
\begin{array}{rll}
 & \mid \boxed{e.x} & \\
 & \mid \boxed{\mathsf{null}} & \\
e_{\mathsf{lit}} ::= & n \mid \mathsf{true} \mid \mathsf{false} \mid \mathit{char} & \text{(constant literals)} \\
\odot ::= & & \text{(binary expressions)} \\
 & \mid + \mid - \mid * \mid *>> \mid / \mid \% \mid & \\
 & \mid == \mid != \mid < \mid \leq \mid > \mid \geq \mid \&\& \mid \mid\mid & \\
\square ::= & - \mid ! & \text{(unary expressions)} \\
t ::= & \mathsf{int} \mid \mathsf{bool} \mid \mathsf{arr}(t) & \text{(type specifiers)} \\
 & \mid \boxed{\mathsf{record}\ r} &
\end{array}
$$

Note that r is the metavariable denoting the name of a record type which shares the same namespace as variables and functions. Now we extend the Eta Type System to include the following (new definitions are also boxed)

$$
\begin{array}{llll}
t ::= \mathsf{int} & T ::= (t_1, t_2, \ldots, t_n)^{n \geq 0} & R ::= \mathsf{unit} & \sigma ::= \mathsf{var}\ t \\
\mid \mathsf{bool} & & \mid \mathsf{void} & \mid \mathsf{ret}\ T \\
\mid t[] & & & \mid \mathsf{fn}\ T \to T' \\
\mid \boxed{\mathsf{record}\ r} & & & \mid \boxed{\mathsf{record}\ \{x_1 : t_1, ..., x_n : t_n\}} \\
\mid \boxed{\mathsf{null}} & & & \mid \boxed{\mathsf{record}\ ?} \\
\mid \mathsf{empty} & & & \mid \boxed{\mathsf{breakable}}
\end{array}
$$

Note that the new symbol table contains two different definitions for record types: $\mathsf{record}\ ?$ and $\mathsf{record}\ \{x_1 : t_1, ..., x_n : t_n\}$. The second represents a record type that is fully defined and the first represents a record type in the first pass of typechecking whose name the typechecker have seen but whose definition has not yet been parsed. In some inference rules, we use the notation $\mathsf{record}\ \rho$ to refer to both definitions. The introduction of new types for definitions requires us to extend the subtyping relation on these new types. According to the Rho Language Specification, the null value is a member of all array and record types, which necessitates the following definition:

$$
\frac{}{\mathsf{null} \leq t[]}\ \textsc{Null-Arr} \qquad \frac{}{\mathsf{null} \leq \mathsf{record}\ r}\ \textsc{Null-Rec} \qquad \frac{}{\mathsf{null} \leq \mathsf{null}}\ \textsc{Null-Refl}
$$

The fact that empty is a subtype of all types and all is a supertype of all types still remains true with the addition of Rho types:

$$
\frac{}{\mathsf{empty} \leq t}\ \textsc{Empty}
$$

We note that, crucially, record types are not related to each other unless they are of the same name:

$$
\frac{}{\mathsf{record}\ r \leq \mathsf{record}\ r}\ \textsc{Rec}
$$

Now we consider constructing typing judgements for expressions in Rho. The form of the judgement $\Gamma \vdash e : t$ remains the same and existing judgement remains valid. First, the null value naturally has the null type:

$$
\frac{}{\Gamma \vdash \mathsf{null} : \mathsf{null}}
$$

Nevertheless, we need to add the following judgements for typing records:

$$
\frac{\Gamma \vdash e : \mathsf{record}\ r \qquad \Gamma(r) = \mathsf{record}\ \{x_1 : t_1, ..., x_n : t_n\}}{\Gamma \vdash e.x_i : t_i} \qquad \frac{}{\Gamma \vdash \mathsf{null}.x : \mathsf{empty}} \qquad \frac{}{\Gamma \vdash \mathsf{empty}.x : \mathsf{empty}}
$$

Now we amend the typing judgements for statements, which also retains the form $\Gamma \vdash s : R \dashv \Gamma'$. First, we add the following judgements to support break statements:

$$
\frac{\Gamma \vdash e : \mathsf{bool} \qquad \Gamma[\beta \mapsto \mathsf{breakable}] \vdash s : R \dashv \Gamma'}{\Gamma \vdash \mathsf{while}\ (e)\ s : \mathsf{unit} \dashv \Gamma} \qquad \frac{\Gamma(\beta) = \mathsf{breakable}}{\Gamma \vdash \mathsf{break} : \mathsf{void} \dashv \Gamma}
$$

Second, since the set of lvalues have been extended, we need to introduce new cases to the helper

judgement $\Gamma, \Gamma' \vdash lv :: t \dashv \Gamma'$:

$$\frac{\Gamma \vdash e : \mathsf{record}\ r \qquad \Gamma(r) = \mathsf{record}\ \{x_1 : t_1, ..., x_n : t_n\}}{\Gamma, \Gamma' \vdash e.x_i : t_i \dashv \Gamma'} \qquad \frac{}{\Gamma, \Gamma' \vdash \mathsf{null}.x : \mathsf{empty} \dashv \Gamma'} \qquad \frac{}{\Gamma, \Gamma' \vdash \mathsf{empty}.x : \mathsf{empty}, \dashv \Gamma'}$$

Another complication that arises at the statement level is that with the introduction of named record types, type specifiers may no longer be well-formed in variable declarations and definitions. Hence, we introduce judgements $\Gamma \vdash t$ and $\Gamma \vdash td$ indicating that the type t or the type specifier td is well-formed:

$$\frac{t \in \{\mathsf{int}, \mathsf{bool}, \mathsf{null}\}}{\Gamma \vdash t} \qquad \frac{\Gamma \vdash t}{\Gamma \vdash \mathsf{arr}(t)} \qquad \frac{\Gamma(r) = \mathsf{record}\ \rho}{\Gamma \vdash \mathsf{record}\ r}$$

The definition for $\Gamma \vdash td$, except we need to handle an extra case $\Gamma \vdash \mathsf{arr}(td, e)$ which has identical logic with the $\Gamma \vdash \mathsf{arr}(td)$ case. Next, we rewrite typing logic for variable declarations as follows, with the additional side conditions highlighted:

$$\frac{x \notin \mathsf{dom}(\Gamma') \qquad \boxed{\Gamma \vdash t}}{\Gamma, \Gamma' \vdash x : td \dashv \Gamma'[x \mapsto \mathsf{var}(\mathsf{type_of}(td))]}$$

where $\mathsf{type_of}(td)$[2] is the type corresponding to the type specifier td in the variable declaration. In the helper judgement defined for the lvalues $\Gamma, \Gamma' \vdash lv :: t \dashv \Gamma'$ for typechecking assignments, we need to make a similar amendment to the case when $lv = x : t$:

$$\frac{x \notin \mathsf{dom}(\Gamma') \qquad \boxed{\Gamma \vdash t}}{\Gamma, \Gamma' \vdash x : t :: t \dashv \Gamma'[x \mapsto \mathsf{var}(t)]}$$

Finally, we consider type-checking toplevel programs. Since the Rho module system have became so much more complex, a complete formal specification of typechecking Rho programs is warranted.

The largest unit of typechecking is a folder in a file system, abstractly represented as a map from filenames to either a program file and interface file, which we denote as $\Pi = \{\mathsf{intf}\, c_1 \mapsto intf_1, ..., \mathsf{intf}\, c_n \mapsto intf_n, \mathsf{prog}\, c'_1 \mapsto prog_1, ..., \mathsf{prog}\, c'_n \mapsto prog_n\}$. Certain programs, but not all, have associated interfaces, in which case $c_i = c'_j$ for some i, j. To define more concise typing rules, The definition for the metavariables representing interfaces $intf$ and programs $prog$ needs to be revised as follows:

$$\begin{array}{lll}
intf ::= gci_1\ ...\ gci_n & & \\
\quad gci ::= & & \text{(global commands for interfaces)} \\
\qquad | \, \mathsf{use}\, c & & \text{(use declarations)} \\
\qquad | \, \mathsf{record}\, r\{x_1 : t_1, ..., x_n : t_n\} & & \text{(record declarations)} \\
\qquad | \, f(x_1 : t_1, ..., x_n : t_n) : t'_1, ..., t'_n & & \text{(function signatures)} \\
\\
prog ::= gcp_1\ ...\ gcp_n & & \\
\quad gcp ::= & & \text{(global commands for programs)} \\
\qquad | \, \mathsf{use}\, c & & \text{(use declarations)} \\
\qquad | \, gd & & \text{(global declarations)} \\
\qquad | \, \mathsf{record}\, r\{x_1 : t_1, ..., x_n : t_n\} & & \text{(record declarations)} \\
\qquad | \, f(x_1 : t_1, ..., x_n : t_n) : t'_1, ..., t'_n\ s & & \text{(function definitions)}
\end{array}$$

and gd is defined in the same way as above. Although AST definition of interfaces and programs does not restrict the order in which each type of global commands could occur for brevity, we require that the global commands are sorted in advance to type checking for both programs and interfaces such that **all uses appear before record declarations before global declarations before function definitions**.

With the above definition, we can now describe how to typecheck interfaces and programs given

[2] we have implemented this function as visitor on type specifiers in declarations.

the folder in which they are stored. First, we need to collect all definitions for functions and types from interfaces and programs, which requires two passes. The first pass has typing judgements of form $\Pi, \Gamma \vdash_1 intf/prog \dashv \Gamma'$ whose sole purpose is to collect the names of all record types. Naturally, derivation of this judgement requires a helper judgement $\Pi, \Gamma \vdash_1 gci/gcd \dashv \Gamma'$ which is also defined below:

$$\frac{}{\Pi, \Gamma \vdash_1 f(x_1 : t_1, ..., x_n : t_n) : t'_1, ..., t'_m \dashv \Gamma} \qquad \frac{}{\Pi, \Gamma \vdash_1 f(x_1 : t_1, ..., x_n : t_n) : t'_1, ..., t'_m \; s \dashv \Gamma}$$

$$\frac{}{\Pi, \Gamma \vdash_1 gd \dashv \Gamma}$$

$$\frac{\Pi(\mathsf{intf}\ c) = intf \qquad \Pi, \Gamma \vdash_1 intf \vdash \Gamma'}{\Pi, \Gamma \vdash_1 \mathsf{use}\ c \dashv \Gamma'} \qquad \frac{\Gamma' = \Gamma[r \mapsto \mathsf{record}\ ?]}{\Pi, \Gamma \vdash_1 \mathsf{record}\ r\{x_1 : t_1, ..., x_n : t_n\} \dashv \Gamma'}$$

$$\frac{\Pi, \Gamma \vdash_1 gci_1 \dashv \Gamma_1 \qquad ..... \qquad \Pi, \Gamma_{n-1} \vdash_1 gci_n \dashv \Gamma_n}{\Pi, \Gamma \vdash_1 gci_1\ ...\ gci_n \dashv \Gamma_n} \qquad \frac{\Pi, \Gamma \vdash_1 gcd_1 \dashv \Gamma_1 \qquad ..... \qquad \Pi, \Gamma_{n-1} \vdash_1 gcd_n \dashv \Gamma_n}{\Pi, \Gamma \vdash_1 gcd_1\ ...\ gcd_n \dashv \Gamma_n}$$

The second pass on interfaces has typing judgements of form $\Pi, \Gamma \vdash_2 intf/prog \dashv \Gamma'$ and checks global declarations, record definitions and function signatures are 1) well formed, 2) have distinct names unless the two definitions agree exactly, before adding them to the typing context. It also requires a helper judgement $\Pi, \Gamma \vdash_2 gci/gcd \dashv \Gamma'$ which is also defined below.

$$\frac{\Gamma \vdash t_i^{\ (\forall i \in 1...n)} \qquad \Gamma \vdash \ t'_i\ ^{(\forall i \in 1...m)} \qquad f \notin \mathsf{dom}(\Gamma) \qquad \Gamma' = \Gamma[f \mapsto \mathsf{fn}(t_1, ..., t_n) \to (t'_1, ..., t'_m)]}{\Pi, \Gamma \vdash_2 f(x_1 : t_1, ..., x_n : t_n) : t'_1, ..., t'_m \dashv \Gamma'}$$

$$\frac{\Gamma(f) = \mathsf{fn}(t_1, ..., t_n) \to (t'_1, ..., t'_m)}{\Pi, \Gamma \vdash_2 f(x_1 : t_1, ..., x_n : t_n) : t'_1, ..., t'_m \dashv \Gamma}$$

$$\frac{\Gamma \vdash t_i^{\ (\forall i \in 1...n)} \qquad \Gamma \vdash \ t'_i\ ^{(\forall i \in 1...m)} \qquad f \notin \mathsf{dom}(\Gamma) \qquad \Gamma' = \Gamma[f \mapsto \mathsf{fn}(t_1, ..., t_n) \to (t'_1, ..., t'_m)]}{\Pi, \Gamma \vdash_2 f(x_1 : t_1, ..., x_n : t_n) : t'_1, ..., t'_m \;\; s \dashv \Gamma'}$$

$$\frac{\Gamma(f) = \mathsf{fn}(t_1, ..., t_n) \to (t'_1, ..., t'_m)}{\Pi, \Gamma \vdash_2 f(x_1 : t_1, ..., x_n : t_n) : t'_1, ..., t'_m \;\; s \dashv \Gamma}$$

$$\frac{\Pi(\mathsf{intf}\ c) = intf \qquad \Pi, \Gamma \vdash_2 intf \vdash \Gamma'}{\Pi, \Gamma \vdash_2 \mathsf{use}\ c \dashv \Gamma'}$$

$$\frac{\Gamma(r) = \mathsf{record}\ ? \qquad f_r \notin \mathsf{dom}(\Gamma) \qquad x_i \neq x_j^{\ (\forall i \neq j)} \qquad \Gamma \vdash t_i^{\ (\forall i \in 1...n)} \qquad \Gamma' = \Gamma[r \mapsto \mathsf{record}\ \{x_1 : t_1, ..., x_n : t_n\}, \; f_r \mapsto \mathsf{fn}(t_1, ..., t_n) \to \mathsf{record}\ r]}{\Pi, \Gamma \vdash_2 \mathsf{record}\ r\{x_1 : t_1, ..., x_n : t_n\} \dashv \Gamma'}$$

$$\frac{\Gamma(r) = \mathsf{record}\ \{x_1 : t_1, ..., x_n : t_n\}}{\Pi, \Gamma \vdash_2 \mathsf{record}\ r\{x_1 : t_1, ..., x_n : t_n\} \dashv \Gamma}$$

$$\frac{\Gamma \vdash t \qquad \Gamma \vdash e : t \qquad x \notin \mathsf{dom}(\Gamma) \qquad \Gamma' = \Gamma[x \mapsto \mathsf{var}\ t]}{\Pi, \Gamma \vdash x : t = e : \Gamma} \qquad \frac{\Gamma \vdash t \qquad x \notin \mathsf{dom}(\Gamma) \qquad \Gamma' = \Gamma[x \mapsto \mathsf{var}\ t]}{\Pi, \Gamma \vdash x : t \;\; : \Gamma'}$$

$$\frac{\Pi, \Gamma \vdash_1 gci_1\ ...\ gci_n \dashv \Gamma' \qquad \Pi, \Gamma' \vdash_2 gci_1 \dashv \Gamma_1 \qquad ..... \qquad \Pi, \Gamma_{n-1} \vdash_2 gci_n \dashv \Gamma_n}{\Pi, \Gamma \vdash_2 gci_1\ ...\ gci_n \dashv \Gamma_n}$$

$$\frac{\Pi, \Gamma \vdash_1 gcd_1\ ...\ gcd_n \dashv \Gamma' \qquad \Pi, \Gamma' \vdash_2 gcd_1 \dashv \Gamma_1 \qquad ..... \qquad \Pi, \Gamma_{n-1} \vdash_2 gcd_n \dashv \Gamma_n}{\Pi, \Gamma \vdash_2 gcd_1\ ...\ gcd_n \dashv \Gamma_n}$$

We discuss the judgement used to add new record definitions in more detail. Notice that the metavariable f_r is used to denote the name of the constructor function for the record r. An astute reader may realize that if we let $f_r = r$, which is precisely the syntax of the source language, a name collision occurs in the typing contet. A workaround for this without creating too much complexity is to convert every reference to a record type in a type specifier to its ABI-compliant name immediately when its parsed, e.g. the record definition record Point{...} becomes record r5Point{...}. In this way, its constructor function still has name Point to be referenced in function calls.

In addition to the two passes described above, programs need an extra pass of typechecking. The third pass has form $\Pi \vdash prog$ ok. As with Eta, this final pass ensures that function bodies are well-typed, but it also performs an conformance check against the program's interface. The third

pass needs a helper judgement $\Pi, \Gamma_1, \Gamma_2 \vdash gcd \dashv \Gamma_3$ defined below. The reason why two separate typing contexts are needed is that Γ_1 tracks the names that the program is visible to and Γ_2 tracks the names that the program itself defines, and only the second of which is used to check conformance.

$$\frac{}{\Pi, \Gamma_1, \Gamma_2 \vdash \mathsf{use}\ c \dashv \Gamma_2}$$

$$\frac{}{\Pi, \Gamma_1, \Gamma_2 \vdash x : t = e \dashv \Gamma_2[x \mapsto \mathsf{var}\ x]} \qquad \frac{}{\Pi, \Gamma_1, \Gamma_2 \vdash x : t \dashv \Gamma_2[x \mapsto \mathsf{var}\ x]}$$

$$\frac{\Gamma_3 = \Gamma_2[r \mapsto \mathsf{record}\ \{x_1 : t_1, ..., x_n : t_n\}, f_r \mapsto \mathsf{fn}(t_1, ..., t_n) \to (\mathsf{record}\, r)]}{\Pi, \Gamma_1, \Gamma_2 \vdash \mathsf{record}\ r\{x_1 : t_1, ..., x_n : t_n\} \dashv \Gamma_3}$$

$$\frac{x_i \notin \Gamma_1^{\forall i \in 1...n} \qquad \Gamma[x_1 \mapsto \mathsf{var}\ t_1, ..., x_n \mapsto \mathsf{var}\ t_n, \rho \mapsto \mathsf{ret}\ (t'_1, ..., t'_m)] \vdash s : \mathsf{void} \dashv \Gamma'_1}{\Pi, \Gamma_1, \Gamma_2 \dashv f(x_1 : t_1, ..., x_n : t_n) : t'_1, ..., t'_m\ s \dashv \Gamma_2[\mathsf{fn}\ (t_1, ..., t_n) \to (t'_1, ..., t'_m)]}$$

$$\frac{x_i \notin \Gamma_1^{\forall i \in 1...n} \qquad \Gamma[x_1 \mapsto \mathsf{var}\ t_1, ..., x_n \mapsto \mathsf{var}\ t_n, \rho \mapsto \mathsf{ret}\ (t'_1, ..., t'_m)] \vdash s : R \dashv \Gamma'_1}{\Pi, \Gamma_1, \Gamma_2 \vdash f(x_1 : t_1, ..., x_n : t_n)\ s \dashv \Gamma_2[\mathsf{fn}\ (t_1, ..., t_n) \to (t'_1, ..., t'_m)]}$$

$$\frac{}{\Pi, \Gamma_1, \Gamma_2 \vdash x : e = t \dashv \Gamma_2[x \mapsto \mathsf{var}\ t]} \quad \frac{}{\Pi, \Gamma_1, \Gamma_2 \vdash x : e = t \dashv \Gamma_2[x \mapsto \mathsf{var}\ t]}$$

Now we give the ultimate judgements for typechecking programs. For brevity, we suppose that the program to be checked is accessed through the filename c.

$$\frac{\Pi, \varnothing \vdash_2 gcd_1\ ...\ gcd_n \dashv \Gamma' \qquad \Pi, \Gamma', \varnothing \vdash gcd_1 \dashv \Gamma_1 \qquad ..... \qquad \Pi, \Gamma', \Gamma_{n-1} \vdash gcd_n \dashv \Gamma_n \qquad \Pi(\mathsf{intf}\ c) = intf \qquad \Pi, \varnothing \vdash_2 intf \dashv \Gamma'_n \qquad \Gamma_n\ \mathsf{conforms\ to}\ \Gamma'_n}{\Pi \vdash prog\ \mathsf{ok}}$$

$$\frac{\Pi, \varnothing \vdash_2 gcd_1\ ...\ gcd_n \dashv \Gamma' \qquad \Pi, \Gamma', \varnothing \vdash gcd_1 \dashv \Gamma_1 \qquad ..... \qquad \Pi, \Gamma', \Gamma_{n-1} \vdash gcd_n \dashv \Gamma_n \qquad \mathsf{intf}\ c \notin \mathsf{dom}(\Pi)}{\Pi \vdash prog\ \mathsf{ok}}$$

We say that Γ conforms to Γ' if and only if:

- if $\Gamma'(f) = \mathsf{fn}\, T \to T'$, then $\Gamma(f) = \mathsf{fn}\, T \to T'$, unless $f_r = r$ for some name of an record type $r \in \mathsf{dom}(\Gamma')$.
- if $\Gamma'(r) = \mathsf{record}\, \{x_1 : t_1, ..., x_n : t_n\}$, then $\Gamma(r) = \mathsf{record}\, \{x_1 : t_1, ..., x_n : t_n, x'_1 : t'_1, ..., x'_n : t'_n\}$ for some names $x'_1, ..., x'_n$ and types $t'_1, ..., t'_n$.